

DATA266 — Lab 1

Generative AI: an LLM from scratch, a sentiment classifier, and a CycleGAN

Team	Lab Pair 33
Members	Shriram Dundigalla · Tejas Nandkishor Sawant
Kaggle team	PairProgramming_Team_33
GitHub repository	https://github.com/SawantTej2912/DATA266_Lab
Report built	02 October 2026, 13:39 PDT

Headline results

Part	Task	Headline metric	Value
1	Character-level GPT from scratch (TinyStories)	Validation cross-entropy (nats/char)	0.7441
2	Yelp Polarity sentiment classification	Best macro-F1 (TextCNN)	0.9344
3	CycleGAN photo ↔ Monet	FID / MiFID (course scorer)	101.44 / 0.4172

A note on how this document was produced

Every number in this report is read at build time from the `metrics_report.csv` that produced it (`scripts/build_report.py`). None is transcribed by hand. This matters because a report is the artifact furthest from the code and so the first to drift: while writing up Part 1 we found the reproducibility manifest still quoting a cross-entropy that a later correction had superseded, and a Part 2 calibration error quoted as 0.0204 where the CSV said 0.020347. Both were caught by `scripts/verify_submission.py`, which re-derives every figure quoted in prose from the CSV behind it and now runs 107 checks across the three parts.

1. Ownership statement

Each part was implemented independently by both members, as Section 2 of the brief requires: we built separate architectures and chose separate hyperparameters, and no model is a near-copy of the other's. What we did share is the *measurement*, deliberately.

Before either of us trained anything we agreed a measurement contract, written down in `TEAM_PROTOCOL.md`: the same scoring code, the same data splits, the same metric definitions and conventions, the same sample sizes. The reason is that a comparison table is only worth reading if the two columns differ because the *models* differ. If one of us reported diversity at word level and the other at character level, or scored FID at a different sample size, the table would compare measurement choices and silently look like a modelling result. Anything in that file is fixed for both of us; anything outside it is each member's own decision and is exactly what the comparison is meant to expose.

Deliverable	Owner
Part 1 — model, training, evaluation, write-up	Each member independently
Part 2 — three models, evaluation, error analysis	Each member independently
Part 3 — CycleGAN, scoring, Kaggle submission	Each member independently
TEAM_PROTOCOL.md — shared measurement contract	Shriram Dundigalla (drafted), both (agreed)

Deliverable	Owner
Repository layout, README, packaging and verification tooling	Shriram Dundigalla
Part 3 human audit — 30 blinded samples, two raters, Cohen's kappa	Both (rater 1: Shriram, rater 2: Tejas)
This report — structure and Part 1–3 individual sections	Shriram Dundigalla; Tejas's sections pending
Per-task comparison tables and joint analysis	Both

At the time of writing Tejas's runs had not been committed to the shared repository, so his columns below read "not yet submitted". They are generated from the same CSV schema as Shriram's and populate automatically once his `metrics_report.csv` is added.

2. Part 1 — A GPT-style language model trained from scratch

2.1 Approach

The task forbids prebuilt Transformer or attention modules, so attention is written out: queries, keys and values as explicit projections, scaled dot-product scores, a lower-triangular boolean mask applied before the softmax, and the head split done by reshaping. Nothing comes from `nn.Transformer`, `nn.TransformerEncoderLayer`, `nn.MultiheadAttention` or `F.scaled_dot_product_attention`. That claim is checked mechanically rather than asserted: the verifier walks the abstract syntax tree of every module and inspects attribute accesses and imports, because the file's own docstring names those modules in order to say they are unused and a text search therefore flags a clean file.

Causal masking is likewise tested rather than described. Perturbing the final token of a sequence and re-running the forward pass changes the logits at that position by 4.54 and at every earlier position by exactly 0.00 — which is what causality means operationally, and is a stronger statement than showing the mask.

Modelling is at **character level** over a 98-symbol vocabulary. The validation split is taken **by story, not by window**: windows are cut inside a story, so splitting at window level would place text from either side of a cut into both halves and leak.

2.2 Architecture and parameters

Architecture	decoder-only GPT, 6 pre-LN blocks, 8 heads, <code>d_model=256</code> , <code>d_ff=1024</code> , <code>block_size=128</code> , learned token + positional embeddings
Parameters	4,822,016
Vocabulary	98 characters
Hyperparameters	AdamW <code>lr=0.0003</code> <code>betas=(0.9, 0.95)</code> <code>wd=0.1</code> <code>dropout=0.1</code> <code>batch=128</code> <code>epochs=12</code> <code>warmup=0.03</code> cosine to 0.1x, <code>grad_clip=1.0</code>
Data	100,000 training / 10,000 validation windows, split by story
Hardware	Apple MPS (arm) — no CUDA device was used

2.3 Results

Metric	Train	Validation
Cross-entropy (nats/char)	0.7089	0.7441
Perplexity (per character)	2.0317	2.1045
Bits per character	1.0227	1.0735
Top-1 next-character accuracy	0.7728	0.7639
Generalization gap	—	0.0352

Training metrics cover the whole training split — 99,968 of 100,000 windows, the remainder being the 32 that `drop_last=True` trims from each epoch. An earlier version computed them from the first 100 batches, 12.8% of the split, and labelled the result "the training loss"; the CSV now carries `train_eval_windows` so the sample size travels with the metric.

Generation metric (T = 0.8)	Mean ± SD over 15 draws
Distinct-2 (word level)	0.9567 ± 0.0295
Repeated 4-gram rate (word level)	0.0017 ± 0.0044
Training-vocabulary word rate	0.9933 ± 0.0081
Best decoding strategy	<code>rep_pen_1.15</code>

The 15 draws are 5 prompts $\times$ 3 seeds. Reporting a spread rather than a single sample changed a conclusion: greedy decoding scores a repeated-4-gram rate of 0.060 on one draw but **0.235 $\pm$ 0.319** over fifteen, so one sample understated the degenerate-looping failure fourfold, and a standard deviation larger than the mean shows the behaviour is bimodal — greedy decoding either escapes the loop or collapses into it depending on the prompt. A repetition penalty of 1.15 over a 64-character window removes repeated 4-grams entirely across all fifteen draws while slightly *raising* the training-vocabulary word rate, so it is not trading fluency for diversity as higher temperature does.

2.4 Stability and cost

NaN / inf count	0	Loss spikes	0
Gradient norm, mean / max	0.701 / 11.128	Training throughput	24,990 tok/s
Total training time	6,144 s	Peak memory	80.0 MB

3. Part 2 — Sentiment classification on Yelp Polarity

3.1 Approach

Three models: a bag-of-words baseline and two experimental architectures that differ from it and from each other in both architecture and hyperparameters. No pretrained embeddings and no pretrained language models are used anywhere — again checked by walking the AST rather than by grepping. All three are evaluated on the **full official 38,000-review test set** (19,000 positive, 19,000 negative), not a subsample.

Preprocessing is shared across all three so the comparison isolates the model: lowercasing, punctuation removal, stopwords removal and Porter stemming, with negation words deliberately retained — removing "not" as a stopword destroys the polarity of a review, which is the one thing the task is about.

3.2 Results on the full test set

Model	Role	Accuracy	Macro-F1	ROC-AUC	MCC	ECE (conf.)	ECE (pos.)	Params
baseline_bow	baseline	0.9245	0.9245	0.9751	0.8492	0.0044	0.0075	3.87M
exp1_bilstm	experimental	0.9334	0.9334	0.9830	0.8671	0.0165	0.0203	6.61M
exp2_textcnn	experimental	0.9344	0.9344	0.9829	0.8688	0.0061	0.0064	5.09M

Two calibration errors are reported because the term "ECE" is ambiguous and the two conventions give different numbers for the same model. **ECE (conf.)** bins the confidence of the *predicted* class over [0.5, 1] — the Guo et al. (2017) formulation. **ECE (pos.)** bins the positive-class probability over [0, 1], which is what a reliability diagram against the observed positive rate actually plots. Reporting one under the other's name is an easy and invisible error; we made it, caught it, and now emit both.

3.3 What the numbers say

Word order helps, but only by about one point. Both experimental models beat the bag-of-words baseline, and the margin is smaller than the effort difference suggests — for polarity detection, which words appear carries most of the signal.

The two experimental models are statistically indistinguishable, and that is the result. Their bootstrap confidence intervals overlap and a paired McNemar test gives $p = 0.396$. There is no evidence either is better, and reporting the 0.001 macro-F1 difference as a ranking would be reading noise. The honest tie-breaker is cost: the TextCNN reaches the same accuracy far more cheaply than the BiLSTM.

The most accurate model is not the best calibrated. The baseline has the best calibration under both definitions despite the worst accuracy, and the BiLSTM the worst calibration at near-best accuracy. Accuracy and calibration are different properties and a system that acts on a probability threshold should be chosen on both.

3.4 Error analysis

Twenty test errors were reviewed by hand in the required composition — five confident false positives, five confident false negatives, five near-threshold errors, and five drawn from the worst-performing slice — each labelled with an error type and one testable fix. A substantial share turn out not to be modelling failures at all: reviews whose star rating contradicts their text, which no classifier can get right and which place a ceiling on achievable accuracy.

4. Part 3 — CycleGAN, photograph ↔ Monet

4.1 Approach and architecture

Generators	U-Net, depth 8, ngf 64, instance normalisation, written from scratch
Discriminators	70×70 PatchGAN, ndf 64
Parameters	114.34M total (G 54.40M each, D 2.76M each)
Objective	LSGAN adversarial + cycle-consistency L1 ($\lambda = 10$) + identity L1 ($\lambda = 5$)
Optimiser	Adam, lr 2e-4, betas (0.5, 0.999); replay buffer of 50; Normal(0, 0.02) initialisation
Run	100 epochs × 1,000 steps = 100,000 generator updates at 256×256, batch size 1, 2.77 h on an NVIDIA A100

A U-Net generator rather than the ResNet the CycleGAN paper uses. The choice was partly a division of labour — the two members of this pair deliberately built different architectures so the comparison in §5.3 measures something — and partly that a U-Net's skip connections carry the input's spatial structure straight across to the decoder, which is what cycle-consistency is asking the generator to preserve anyway. The cost is that those same skips make it easy for the generator to pass content through unchanged and satisfy the cycle loss without doing any translation, which is exactly what the content-cosine figure in §4.2 is there to detect.

The domains are badly unbalanced — 7,038 photographs against 300 Monet paintings — so an epoch is defined as a fixed 1,000 steps that draw from both domains independently rather than as one pass over the smaller domain. Over 100 epochs that is 100,000 draws against 7,038 photographs, so the photograph domain is covered many times over and the Monet domain is heavily repeated. The repetition on the small side is the structural risk in this task: it is what lets a discriminator memorise the target domain, and it is why §4.3 reports precision and recall separately rather than FID alone.

4.2 Final results

All FID and MiFID figures below come from the course evaluation script, reproduced as `evaluate_local.py` with its paths exposed as arguments. That script caps each side at 300 images, normalises with ImageNet statistics rather than to $[-1, 1]$, recomputes its reference statistics from the real image folders, and reports the mean of both directions. A FID computed under any other convention is on a different scale and is not comparable to these numbers or to the leaderboard.

FID A→B	103.92	FID B→A	98.95
FID averaged (submitted)	101.44	MiFID averaged (submitted)	0.4172
KID averaged	0.0136	LPIPS averaged	0.2152
Cycle reconstruction L1	0.0482	Content cosine	0.7756
Leaderboard rank	20	Peak GPU memory	1882 MB

The two directions are not symmetric and the asymmetry is informative. Photograph → Monet scores better on FID, which is the easier direction: the target is a 300-image distribution with a narrow, consistent style, so a generator that learns the palette and the brushwork lands close to it. Monet → photograph has to hit the full variety of 7,038 photographs and scores worse. The precision and recall figures in §4.3 show the same split from the other side.

4.3 Precision, recall, density and coverage

FID collapses fidelity and diversity into one number, so a model that produces a few excellent images and a model that produces many mediocre ones can score alike. These four report the two axes separately:

Direction	Precision	Recall	Density	Coverage
A→B (Monet → photo)	0.6833	0.4433	0.8627	0.7900
B→A (photo → Monet)	0.5000	0.7300	0.3847	0.6933

A→B has high precision and low recall: its outputs look like photographs but cover only part of what photographs look like. B→A is the mirror image — lower precision, higher recall — which is the signature of a generator producing varied output that is not yet convincingly Monet. Neither direction shows the precision-near-one, recall-near-zero pattern that indicates mode collapse, and no non-finite loss occurred in any of the three runs.

4.4 Integrity

Every submitted image is the direct output of our own trained CycleGAN. No image was hand-picked, manually edited, copied, or sourced externally; no outputs are hardcoded or looked up; no test-set pairings were used or inspected during training; and no pretrained or foundation image model was used to generate or modify any submitted image. The generators are initialised from $\text{Normal}(0, 0.02)$ and trained from that initialisation only. Pretrained Inception and AlexNet networks appear solely as *scorers*, for FID/KID and LPIPS respectively, as the rubric requires. The submitted directories are verified to hold 300 and 7,038 JPEG images at 256×256 whose filenames match their source domains one-to-one.

5. Per-task comparison across team members

These tables compare the two members' independently built models on identical measurements, per the contract in TEAM_PROTOCOL.md. Tejas's runs were not in the shared repository at build time; the rows populate automatically from his metrics_report.csv once added.

5.1 Part 1 — language model

Metric	Shriram Dundigalla	Tejas Nandkishor Sawant
Architecture	decoder-only GPT, 6 pre-LN blocks, 8 heads, d_model=256, d_f...	not yet submitted
Parameters	4,822,016	not yet submitted
Validation cross-entropy	0.7441	not yet submitted
Validation perplexity	2.1045	not yet submitted
Validation bits/char	1.0735	not yet submitted
Top-1 accuracy	0.7639	not yet submitted
Generalization gap	0.0352	not yet submitted

5.2 Part 2 — sentiment classification (best model per member)

Metric	Shriram Dundigalla	Tejas Nandkishor Sawant
Best model	exp2_textcnn	not yet submitted
Accuracy	0.9344	not yet submitted
Macro-F1	0.9344	not yet submitted
ROC-AUC	0.9829	not yet submitted
MCC	0.8688	not yet submitted
ECE (positive class)	0.0064	not yet submitted
Parameters	5.09M	not yet submitted

5.3 Part 3 — CycleGAN

Metric	Shriram Dundigalla	Tejas Nandkishor Sawant
FID (course scorer, averaged)	101.44	not yet submitted
MiFID (averaged)	0.4172	not yet submitted
KID (averaged)	0.0136	not yet submitted
Cycle reconstruction L1	0.0482	not yet submitted
Content cosine	0.7756	not yet submitted
LPIPS (averaged)	0.2152	not yet submitted

6. Joint analysis — strengths, weaknesses, limitations, next steps

6.1 Part 1 — language model

Strengths	Attention is implemented from first principles and its causality is verified numerically rather than argued. Training was completely stable: zero NaNs, zero loss spikes, mean gradient norm 0.70 against a clip of 1.0. The validation split is by story, so no text leaks across it.
Weaknesses	The model underfits. Validation loss was still falling monotonically at the final epoch and the generalization gap is only 0.035 nats, so the run stopped because a laptop ran out of hours, not because the model converged. At character level with a 128-character window the model also loses track of its own protagonist across a few sentences.
Limitations	4.8M parameters and a 128-character context against 12.8M characters of text; a 20,000-story subset of TinyStories rather than the full corpus; and all training on an Apple M4 via MPS, which caps the achievable run length.
Next steps	Both fixes are configured and smoke-tested as GPU presets rather than left as aspirations: one trains the same architecture for 30 epochs at a 256-character context, the other scales to 25.4M parameters under an otherwise identical recipe so the difference between them isolates capacity. They are not run here because they are 9–15 and 25–33 hours respectively on MPS.

6.2 Part 2 — sentiment classification

Strengths	All three models are evaluated on the complete 38,000-review test set with bootstrap confidence intervals and paired McNemar tests, so claims of difference are tested rather than asserted. Slice metrics locate where the gains actually land. Both ECE conventions are reported under distinct names.
Weaknesses	The two experimental models are statistically indistinguishable, so the experiment does not identify a winner on accuracy. The BiLSTM costs roughly 90× the training time of the baseline for about one point of accuracy, and is the worst calibrated of the three.
Limitations	Single seed per model, so run-to-run variance is not separated from architectural difference. Reviews are truncated at 250 tokens. Vocabulary is capped at 30,000 types, and the resulting out-of-vocabulary rate is itself a predictor of error.
Next steps	Repeat each model across several seeds and report the spread, so the 'statistically indistinguishable' conclusion rests on between-seed variance as well as between-model. Apply temperature scaling on a validation split to fix the BiLSTM's calibration without touching its accuracy.

6.3 Part 3 — CycleGAN

Strengths	The reported score is produced by the course evaluation script itself, reproduced here with its paths exposed as arguments and checked line for line against the original on the five choices that move the number: the normalisation, the resize, the crop, the Inception input transform, and the distance used for MiFID. The script is validated by scoring the real image folders against themselves, which returns FID -0.000 and MiFID 0.0000. Beyond FID we report precision, recall, density, coverage, KID and LPIPS, because FID alone cannot distinguish a narrow-but-accurate generator from a broad-but-mediocre one, and here the two directions sit on opposite sides of that split.
Weaknesses	We initially scored with our own harness, against the competition's shipped reference statistics under a $[-1, 1]$ convention. That reading was defensible and self-consistent -- scoring the real Monets as if generated returned 0.069 against the shipped statistics, versus 20.7 and 27.6 for the alternatives, so the convention was right about how those statistics were built. It was still the wrong scale for the grade, because the course script recomputes its reference from the image folders under ImageNet normalisation at $n = 300$. Every FID we had recorded before finding this was uncomparable to the leaderboard. The fix was to stop maintaining a second scorer at all.
Limitations	The limiting factor on the score is the 300-image target domain, not the schedule: 100,000 generator updates is a full run and the loss curves are flat well before the end, so more of the same training is not what is missing. The U-Net's skip connections are the architectural tension -- they make cycle consistency easy to satisfy without translating, and a content cosine of 0.78 says a substantial amount of input structure survives to the output, which is partly correct behaviour and partly the shortcut.
Next steps	Differentiable augmentation on the discriminator is the standard answer to a small target domain and we tried it, expecting an improvement. It made things worse -- FID 112.97 against 101.44 -- with the generator gradient norm rising from a mean of 27 to a mean of 137. The policy was too aggressive for a run already at 100,000 updates. The honest next step is a weaker policy under a matched update budget, not the conclusion that the technique does not work.

7. Evidence index

Every number in this report traces to a file in the repository. Paths are relative to the repository root; per-task paths in the second half of the table are relative to that task's member directory.

Claim / metric group	Evidence in the repository
Part 1 — all reported metrics	task1_llm/shriram_dundigalla/metrics_report.csv
Part 1 — training run, unedited log	reproducibility/raw_logs/task1_gpt_char_tinystories_v1_20260928T204206Z.log
Part 1 — weights	task1_llm/shriram_dundigalla/checkpoints/gpt_char_tinystories_v1_20260928T204206Z.pt
Part 1 — loss and validation curves	task1_llm/shriram_dundigalla/outputs/loss_curves_*.png
Part 1 — generation spread and decoding comparison	outputs/generation_sweep_*.csv, outputs/decoding_comparison_*.csv
Part 2 — all reported metrics	task2_sentiment/shriram_dundigalla/metrics_report.csv
Part 2 — significance tests and slices	outputs/mcnemar_tests.csv, outputs/slice_metrics.csv
Part 2 — twenty reviewed errors	outputs/error_review_exp2_textcnn.csv, failure_analysis.md
Part 2 — weights (three models)	task2_sentiment/shriram_dundigalla/checkpoints/*.pt
Part 3 — all reported metrics, all tags	task3_gan/shriram_dundigalla/metrics_report.csv
Part 3 — leaderboard submission	task3_gan/shriram_dundigalla/submission.csv
Part 3 — the scorer is the course script, unmodified	task3_gan/Part3_Evaluation_Script.ipynb vs shriram_dundigalla/evaluate_local.py
Part 3 — the scorer is correct (real vs real)	FID -0.000, MiFID 0.0000
Part 3 — training notebook and checkpoint	task3_gan/shriram_dundigalla/src/task3_cyclegan_unet.ipynb, checkpoints/ckpt_epoch060.pth
All parts — environment and checkpoint-to-result mapping	reproducibility/manifests/shriram_dundigalla_manifest.md
All parts — every quoted figure re-derived from its CSV	scripts/verify_submission.py (107 checks)

8. Reproducing any result

The repository is config-driven throughout: no personal file paths, credentials or API keys appear in any committed file, and every run is specified by a YAML config whose paths are relative to the repository root. A grader can reproduce any single run with one documented command from the README, and can check the whole submission's internal consistency with `python scripts/verify_submission.py`.

9. References

Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.

Eldan, R. and Li, Y. (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759.

Zhu, J.-Y., Park, T., Isola, P. and Efros, A. (2017). *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. ICCV.

Heusel, M. et al. (2017). *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. NeurIPS. (FID)

Binkowski, M. et al. (2018). *Demystifying MMD GANs*. ICLR. (KID)

Zhao, S. et al. (2020). *Differentiable Augmentation for Data-Efficient GAN Training*. NeurIPS. (DiffAugment)

Zhang, R. et al. (2018). *The Unreasonable Effectiveness of Deep Features as a Perceptual Metric*. CVPR. (LPIPS)

Guo, C. et al. (2017). *On Calibration of Modern Neural Networks*. ICML. (ECE)

Holtzman, A. et al. (2020). *The Curious Case of Neural Text Degeneration*. ICLR. (nucleus sampling)

Keskar, N. et al. (2019). *CTRL: A Conditional Transformer Language Model for Controllable Generation*. arXiv:1909.05858. (repetition penalty)